

CAPSTONE PROJECT

AN ML APPROACH TO EMPLOYEE SALARY PREDICTION

Presented By:

1. Student Name- Uttam Singh Chaudhary
2. AICTE ID- STU615be915276d31633413397
3. IBM Skill Build Id:- uttamsinghchaudhary62@gmail.com

OUTLINE

- **Problem Statement** (Should not include solution)
- **System Development Approach** (Technology Used)
- **Algorithm & Deployment (Step by Step Procedure)**
- **Result**
- **Conclusion**
- **Future Scope(Optional)**
- **References**

PROBLEM STATEMENT

- **Problem:** Accurately predicting individual income levels is complex due to diverse demographic and employment factors.
- **Motivation:** The ability to predict income has significant real-world implications for workforce planning, resource allocation, and understanding socio-economic disparities.
- **Challenge:** Handling categorical data and identifying the most influential features in the dataset are key challenges.
- **Goal:** To build effective predictive models for classifying income levels ($>50K$ or $\leq 50K$).
- **Purpose:** To demonstrate ML application on a real-world classification problem and gain insights into income-determining factors.

SYSTEM APPROACH

This project employs a machine learning-based approach to predict employee salary. The development and implementation strategy involved data preprocessing, model building, evaluation, and deployment.

System Requirements

- **RAM:** At least 8GB (for handling the dataset and running models)
- **Storage (ROM):** Sufficient space to store the dataset, code, and generated model files (e.g., a few GB)
- **Processing Power:** A multi-core processor is recommended for faster model training. Access to a GPU can significantly accelerate training for certain models (though not strictly required for this project).

LIBRARIES USED AND THEIR PURPOSE

1. **Pandas:** Used for data manipulation and analysis, including reading the dataset, handling missing values, dropping duplicates, and transforming data.
2. **Numpy:** Used for numerical operations, particularly for handling arrays and mathematical functions.
3. **Matplotlib:** Used for creating static, interactive, and animated visualizations in Python.
4. **Seaborn:** A statistical data visualization library based on matplotlib, used for creating informative and attractive statistical graphics.
5. **StandardScaler:** Used for standardizing features by removing the mean and scaling to unit variance.

LIBRARIES USED AND THEIR PURPOSE

6. **Sklearn.pipeline.Pipeline:** Used to chain multiple estimators into a single estimator.
7. **Sklearn:** Used for building different ML models like logistic regression, DecisionTreeClassifier, RandomForestClassifier, XGBClassifier.
8. **Sklearn.metrics:** Used to evaluate the model performance such as accuracy, precision, recall, f1score, Classification Report.
9. **Pickle:** Used for serializing and deserializing Python object structures, used here for saving and loading the trained model.
10. **Joblib:** Used for saving and loading Python objects that are especially large numpy arrays, used here for saving and loading the trained model.

LIBRARIES USED AND THEIR PURPOSE

- 11. **Streamlit:** Used for creating a web application to deploy the trained model.
- 12. **Pyngroak:** Used to create a public URL for the locally running Streamlit application.
- 13. **OS:** Used for interacting with the operating system, specifically for running shell commands.
- 14. **Threading:** Used for running the Streamlit application in a separate thread.
- 15. **Time:** Used for adding delays in the execution

ALGORITHM & DEPLOYMENT

This step-by-step process covers the entire machine learning workflow, from understanding the data to deploying a functional application.

Here's a breakdown of the key stages in this project:

- 1. Data Loading and Initial Exploration:** The project begins by loading the dataset into a pandas DataFrame. Initial checks are performed using `df.info()` and `df.describe()` to understand the data types, non-null values, and basic statistics. Missing values (represented as '?') and duplicate entries are identified.
- 2. Data Cleaning and Preprocessing:** Duplicate rows are removed to ensure data integrity. Missing values in categorical columns like 'workclass' and 'occupation' are addressed by replacing '?' with 'Others'. Certain rows with 'workclass' values that are not relevant to salary prediction ('Never-worked', 'Without-pay') are dropped. The 'education' column is dropped as it is redundant, with 'educational-num' providing similar information. Categorical features are encoded into numerical representations using `LabelEncoder` to prepare the data for machine learning models.

ALGORITHM & DEPLOYMENT

- 3. Exploratory Data Analysis (EDA):** Visualizations are created to understand the distribution of key features like 'workclass', 'occupation', 'gender', 'income', and 'age'. A correlation heatmap is generated to visualize the relationships between numerical features and the target variable ('income'). This helps in identifying features that are less correlated with income and can be considered for removal.
- 4. Feature Selection:** Based on the correlation analysis, features with low correlation to the 'income' column ('workclass', 'fnlwgt', 'occupation', 'race', 'native-country') are dropped to simplify the model and potentially improve performance.

ALGORITHM & DEPLOYMENT

5. **Model Building and Evaluation:** The data is split into training and testing sets. Several machine learning classification models are trained on the data:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

For models other than XGBoost, StandardScaler is applied within a Pipeline to standardize the features before training. Each model's performance is evaluated using metrics such as accuracy, precision, recall, and F1-score.

A classification report was generated for a detailed evaluation of each model's performance on the test set.

ALGORITHM & DEPLOYMENT

6. **Model Selection:** The models are compared based on their accuracy scores. The model with the highest accuracy is identified as the "Champion model" (in this case, RandomForest).
7. **Model Saving:** The trained "Champion model" is saved using joblib for later use in deployment.
8. **Deployment (Streamlit App):**
 - A streamlit application (app.py) is created to provide a user interface for the model.
 - The saved model is loaded into the Streamlit app.
 - The app allows users to input employee details through a web form for single predictions.
 - It also includes a batch prediction feature where users can upload a CSV file for predictions on multiple entries.
 - The predictions are displayed in the app, and for batch predictions, the results can be downloaded as a CSV file.
 - pyngrok is used to create a public URL, making the locally hosted Streamlit app accessible over the internet.

RESULT

```
1 # Gather basic information about the data
2 df0.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 48842 entries, 0 to 48841
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	age	48842 non-null	int64
1	workclass	48842 non-null	object
2	fnlwgt	48842 non-null	int64
3	education	48842 non-null	object
4	educational-num	48842 non-null	int64
5	marital-status	48842 non-null	object
6	occupation	48842 non-null	object
7	relationship	48842 non-null	object
8	race	48842 non-null	object
9	gender	48842 non-null	object
10	capital-gain	48842 non-null	int64
11	capital-loss	48842 non-null	int64
12	hours-per-week	48842 non-null	int64
13	native-country	48842 non-null	object
14	income	48842 non-null	object

```
dtypes: int64(6), object(9)
```

```
memory usage: 5.6+ MB
```

```
1 # Display value count in 'workclass' column
2 df1['workclass'].value_counts()
```



workclass		count
Private		33860
Self-emp-not-inc		3861
Local-gov		3136
?		2795
State-gov		1981
Self-emp-inc		1694
Federal-gov		1432
Without-pay		21
Never-worked		10

```
dtype: int64
```

```
1 # check count of 'race' in data.
2 df1['race'].value_counts()
```



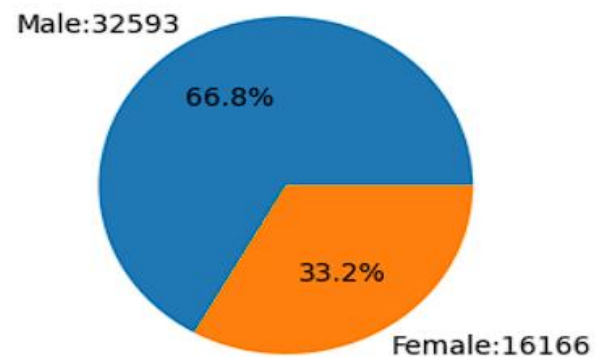
race		count
White		41688
Black		4679
Asian-Pac-Islander		1516
Amer-Indian-Eskimo		470
Other		406

```
dtype: int64
```

GRAPHS

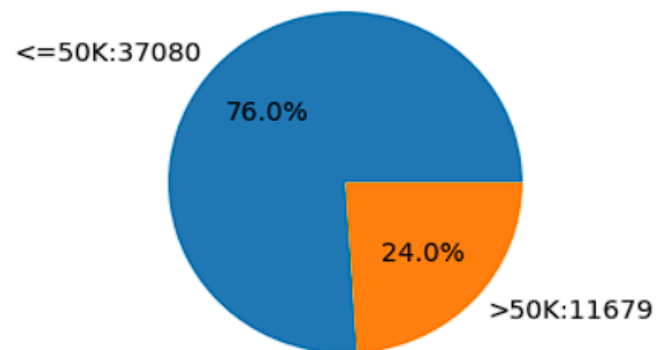
Text(0.5, 1.0, 'Gender Distribution')

Gender Distribution



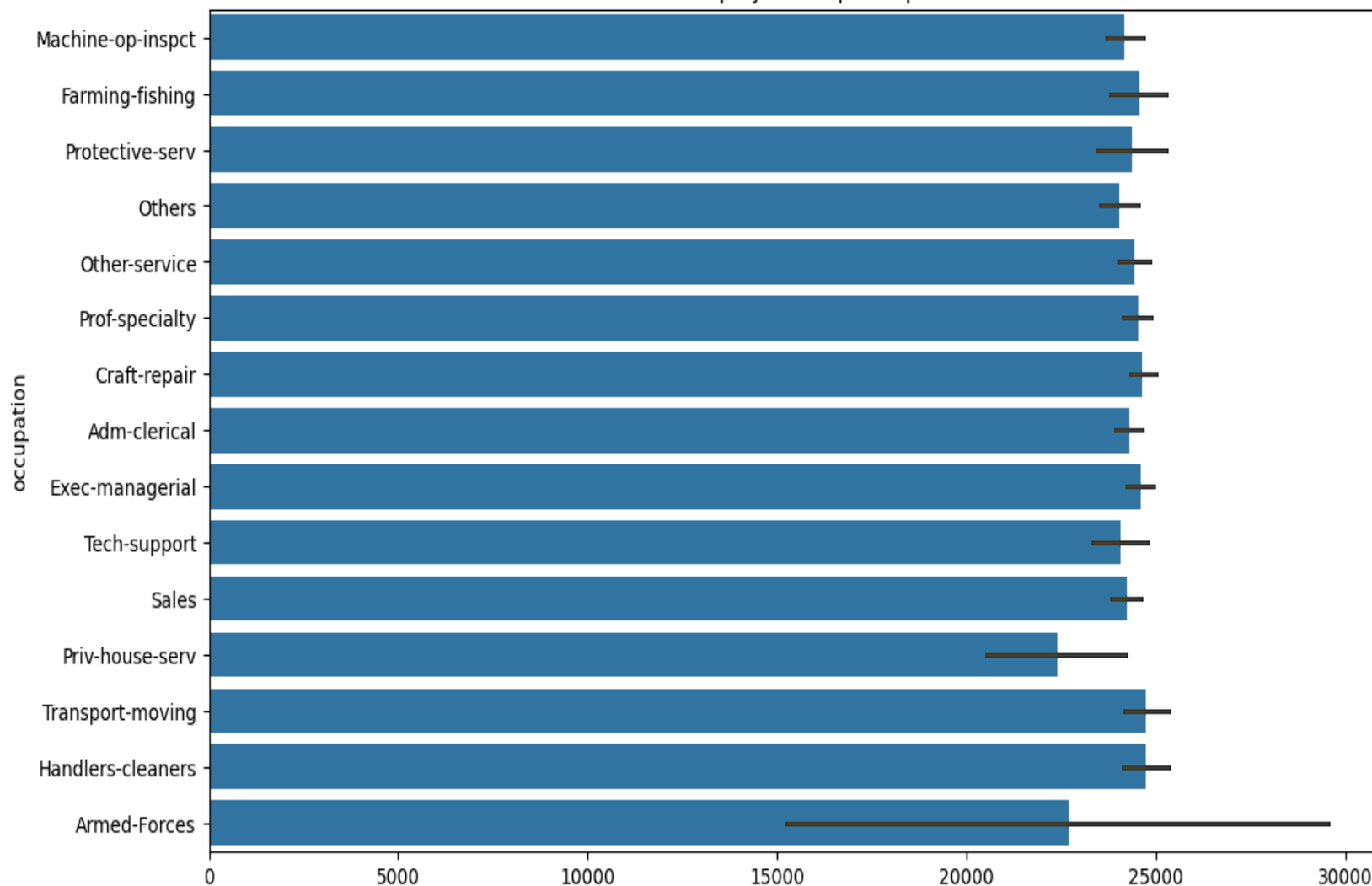
Text(0.5, 1.0, 'Income Distribution Analysis')

Income Distribution Analysis

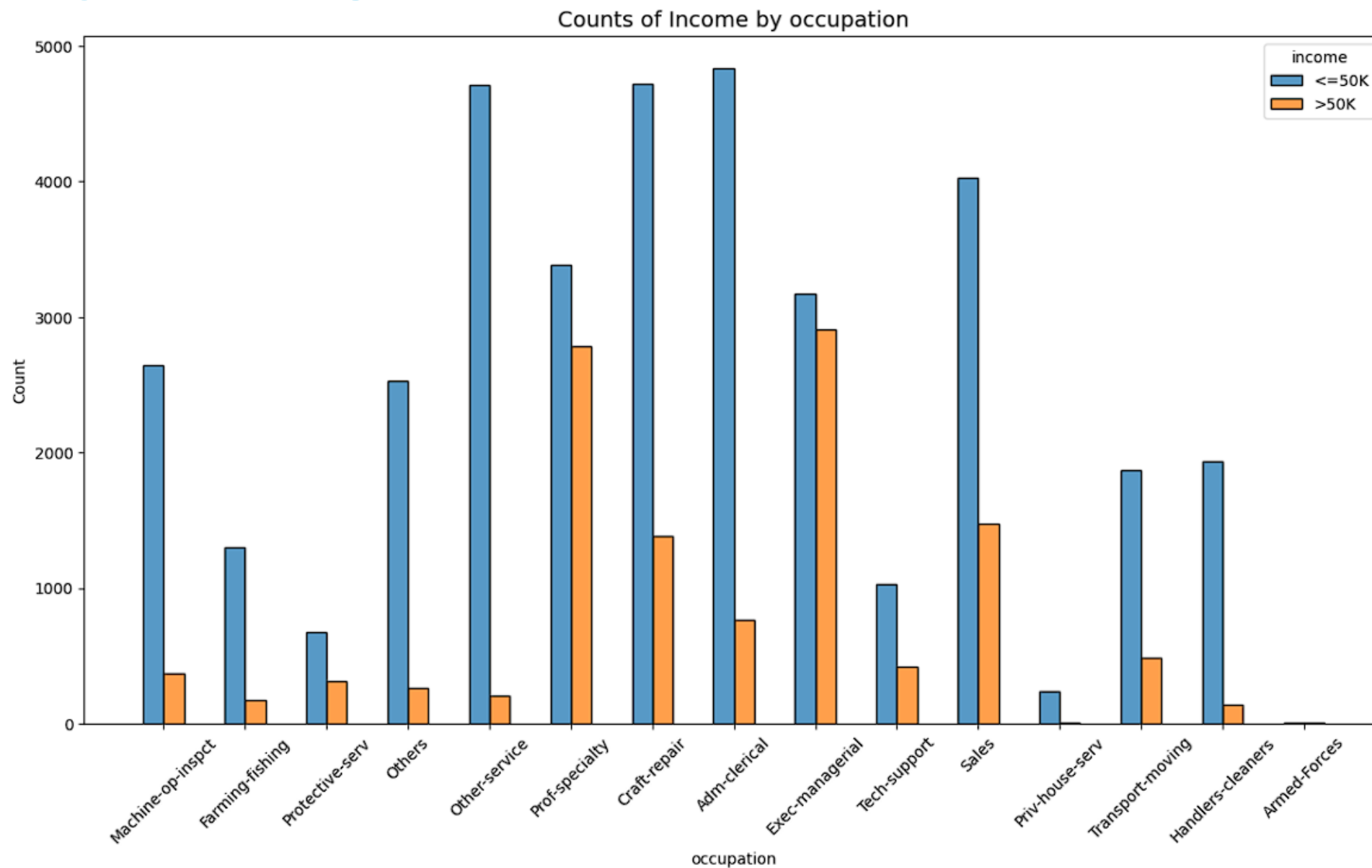


Text(0.5, 1.0, 'Count of Employees as per department')

Count of Employees as per department



GRAPHS

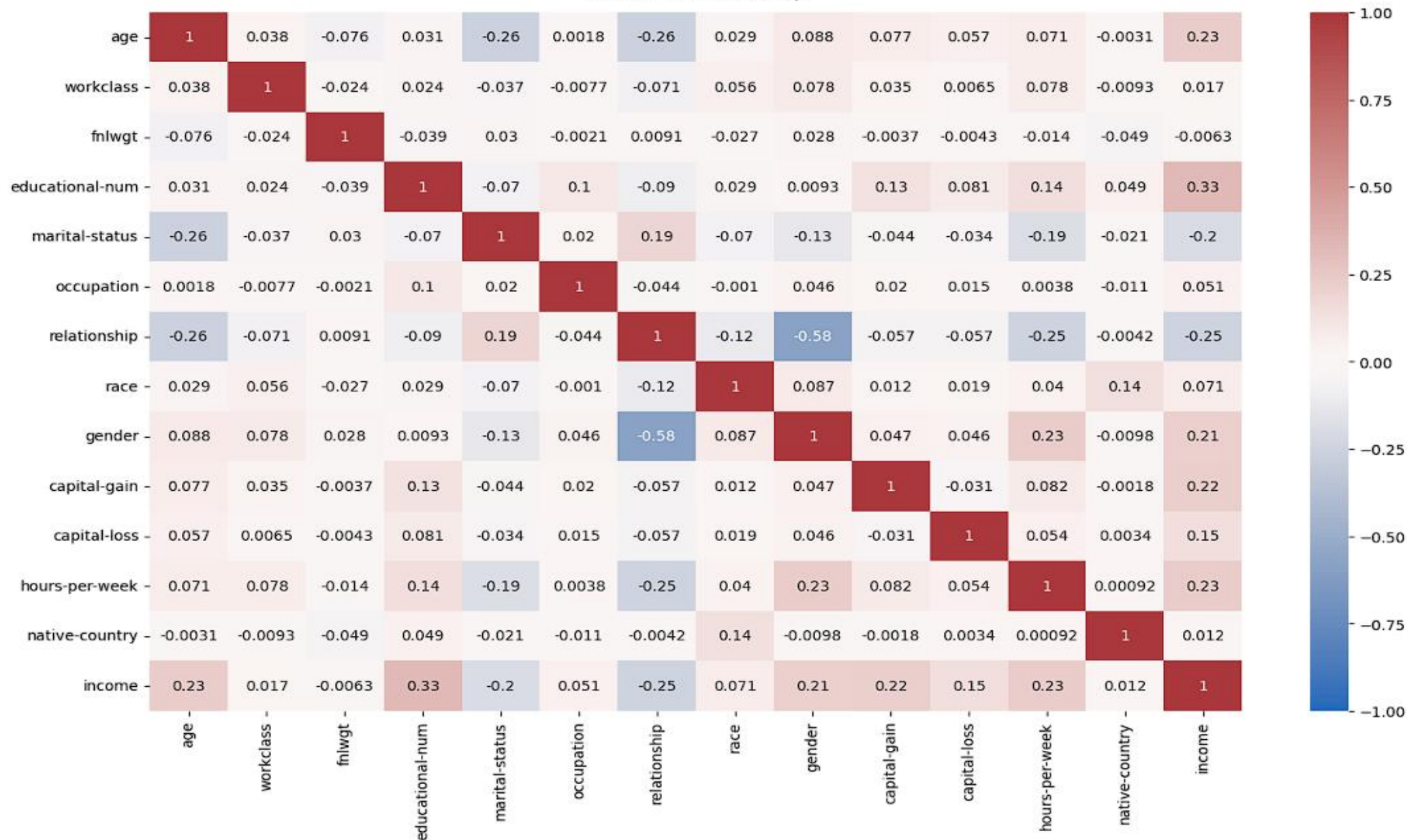


```
Text(0.5, 1.0, 'Age Distribution of Employees')
```





Correlation Heatmap





Logistic Regression Accuracy: 0.8183

	precision	recall	f1-score	support
0	0.84	0.94	0.89	9270
1	0.69	0.44	0.53	2920
accuracy			0.82	12190
macro avg	0.77	0.69	0.71	12190
weighted avg	0.81	0.82	0.80	12190

Decision Tree Accuracy: 0.8280

	precision	recall	f1-score	support
0	0.87	0.91	0.89	9270
1	0.66	0.57	0.61	2920
accuracy			0.83	12190
macro avg	0.77	0.74	0.75	12190
weighted avg	0.82	0.83	0.82	12190

RandomForest Accuracy: 0.8433

	precision	recall	f1-score	support
0	0.88	0.92	0.90	9270
1	0.70	0.60	0.65	2920
accuracy			0.84	12190
macro avg	0.79	0.76	0.77	12190
weighted avg	0.84	0.84	0.84	12190

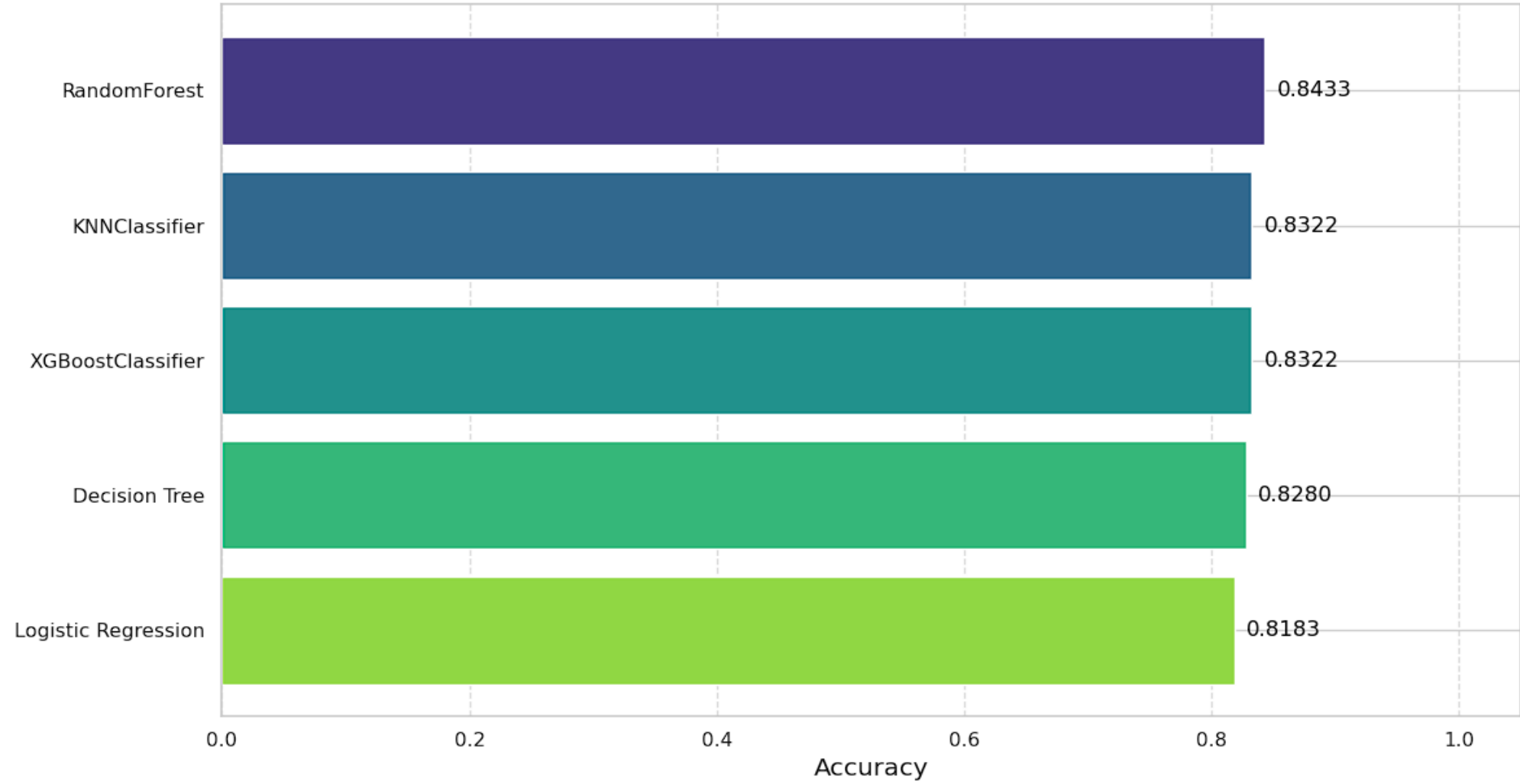
KNNClassifier Accuracy: 0.8322

	precision	recall	f1-score	support
0	0.88	0.91	0.89	9270
1	0.67	0.59	0.63	2920
accuracy			0.83	12190
macro avg	0.77	0.75	0.76	12190
weighted avg	0.83	0.83	0.83	12190

XGBoostClassifier Accuracy: 0.8322

	precision	recall	f1-score	support
0	0.88	0.91	0.89	9270
1	0.67	0.59	0.63	2920
accuracy			0.83	12190
macro avg	0.77	0.75	0.76	12190
weighted avg	0.83	0.83	0.83	12190

Model Accuracy Comparison



GITHUB LINK

Home | Session Recordings - OneDrive | employee_salary_prediction | give link of all this :- panda | pyngrok - PyPI | Employee Salary Classification

https://8d33ee3465b7.ngrok-free.app

Enter Employee Details

Age: 30 (Range: 18 to 65)

Education Level: Bachelors

Occupation: Tech-support

Hours per Week: 40 (Range: 1 to 80)

Years of Experience: 5 (Range: 0 to 40)

Employee Salary Classification

Use this app to predict whether an employee earns >50K or ≤50K based on various inputs.

Preview of Input Data

	age	education	occupation	hours-per-week	experience
0	30	Bachelors	Tech-support	40	5

Predict Salary Class

Batch Prediction (Upload CSV)

Upload a CSV file with employee data

Drag and drop file here
Limit 200MB per file • CSV

Browse files

Streamlit App Link:- <https://8d33ee3465b7.ngrok-free.app/>

GitHub Link:- https://github.com/ut-si-ch/Employee_Salary_Prediction_Uttam_AICTE_EDUNET.git

CONCLUSION

Project Summary

- Successfully loaded, cleaned, and preprocessed the employee income dataset.
- Explored key features and their distributions through visualizations.
- Built and evaluated several machine learning models for income classification.
- Identified the Random Forest Classifier as the best-performing model based on accuracy.

Challenges Faced

- Dealing with missing values represented by non-standard characters ('?') required careful handling.
- Identifying and deciding on which features to drop based on correlation requires analysis to avoid losing important information.

CONCLUSION

Why the Champion Model Performed Well

- **Ensemble Method:** Random Forest is an ensemble learning method that combines multiple decision trees. This reduces overfitting and improves generalization compared to individual decision trees.
- **Feature Importance:** The model effectively leveraged the important features identified during the analysis (such as educational-num, age, hours-per-week, and capital-gain) to make predictions.
- **Robustness:** Random Forests are generally robust to outliers and noisy data, which might have been present in the dataset despite cleaning efforts.
- **Handling of Non-linear Relationships:** Random Forest can capture complex, non-linear relationships between features and the target variable, which is likely present in income prediction.

In conclusion, the Random Forest model provided the most accurate predictions for employee income classification, demonstrating the effectiveness of ensemble methods for this type of problem. The deployed Streamlit application allows for practical use of the trained model.

FUTURE SCOPE(OPTIONAL)

Future Scope

- **Explore Advanced Models:** Investigate deep learning models or more complex ensemble techniques for potential improvements in accuracy.
- **Incorporate External Data:** Integrate additional relevant datasets (e.g., economic indicators, industry trends) to enrich the features and potentially enhance predictive power.
- **Hyperparameter Tuning and Cross-Validation:** Implement more extensive hyperparameter tuning and cross-validation techniques for all models to optimize their performance rigorously.
- **Feature Engineering:** Create new features from existing ones (e.g., interaction terms, polynomial features) to potentially provide more informative inputs for the models.

REFERENCES

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Dataset Source:- <https://archive.ics.uci.edu/dataset/2/adult>
- [pandas](#) | [scikit-learn](#) | [XGBoost](#) | [matplotlib](#) | [seaborn](#) | [streamlit](#) | [pyngrok](#)



THANK YOU